

TECHNICAL DOCUMENTATION

Links Resource

Premium resource directory & digital marketing
agency platform — Next.js 16 rebuild of
linksresource.com

Version 0.1.0

June 2026

linksresource.com

Table of Contents

1. Executive Summary	03
2. Tech Stack	04
3. System Architecture	05-06
4. Site Map	07
5. Service Pages	08
6. User Flows	09-10
7. Data Layer	11
8. Prisma Models	12-13
9. SEO	14
10. AI Chat Assistant	15
11. Admin Dashboard	16
12. API Routes	17
13. Components	18
14. Environment Variables	19
15. Deployment	19
16. Roadmap	20
17. File Structure	20

1. Executive Summary

Links Resource is a premium dark-themed resource directory and digital marketing agency platform. It rebuilds the original linksresource.com experience with a modern Next.js 16 stack, preserving all agency content — services, case studies, reviews, FAQ, pricing, and contact — in a structured data layer.

The site runs fully on an in-memory data layer (`src/lib/data.ts`) with **no database required** for browsing, search, and AI chat. PostgreSQL + Prisma is optional and enables user accounts, favorites, submissions, and admin persistence.

Primary Goals

- Showcase Links Resource agency services with rich landing pages and case studies
- Provide a curated directory of SEO, link building, content, design, and AI tools
- Deliver instant search, category/tag filtering, and an AI recommendation assistant
- Support admin analytics and content management when a database is connected
- Maximize SEO with dynamic metadata, JSON-LD, sitemap, and Open Graph

Key Features

Animated Hero

Typing effect, particles, mouse parallax, scroll fade — full-screen first impression.

AI Assistant

Floating chat widget with streaming responses and built-in recommendation engine.

Resource Explorer

Instant search, category/pricing filters, sort by rating, views, or date.

Service Landings

Seven dedicated agency service pages with packages, process, FAQ, and CTAs.

Admin Dashboard

Analytics overview, resource/category/tag/user management, SEO settings.

Design System

Glassmorphism, gradient accents, responsive layout, reduced-motion support.

Site Identity

Property	Value
Legal name	Links Resource LTD
Domain	linksresource.com
Tagline	Premium Link Building & Digital Marketing Agency
Location	Glasgow, United Kingdom

2. Tech Stack

The project uses a modern React/Next.js stack optimized for static generation, server components, and progressive enhancement.

Next.js 16

App Router, React Server Components, Server Actions, middleware, dynamic OG icons, and static params for resource pages.

React 19

Latest React with concurrent features. Client components used for interactivity (forms, AI chat, animations).

TypeScript 5

Strict typing across data models, API routes, and component props. Shared types in `src/lib/types.ts`.

Tailwind CSS v4

CSS-based theme configuration in `globals.css`. Custom gradients, glass effects, and dark palette.

Framer Motion 12

Scroll reveal, page transitions, AI panel animations, and interactive UI micro-interactions.

GSAP 3

Advanced hero animations and scroll-linked effects on the homepage.

PostgreSQL + Prisma 6

Optional persistence layer. Docker Compose on port 5433. Schema covers users, resources, favorites, submissions.

NextAuth v5

Credentials provider + optional Google/GitHub OAuth. Prisma adapter for session storage.

UI & Utilities

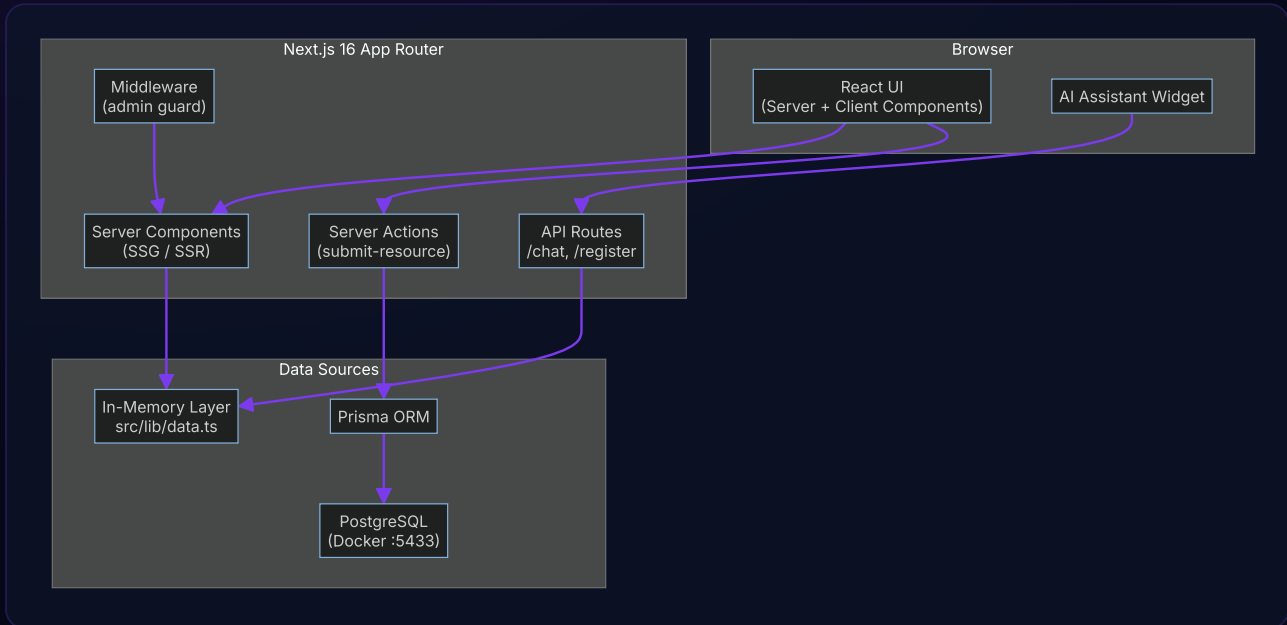
[Radix UI Slot](#)[Lucide Icons](#)[class-variance-authority](#)[clsx + tailwind-merge](#)[Sonner \(toasts\)](#)[next-themes](#)[Zod 4](#)[bcryptjs](#)

Dev Tooling

- **ESLint** — eslint-config-next for App Router conventions
- **tsx** — TypeScript execution for Prisma seed script
- **Puppeteer-core** — PDF generation from this documentation HTML
- **Docker Compose** — Local PostgreSQL 16 container

3. System Architecture

Links Resource follows a layered Next.js App Router architecture. Static content is served from the in-memory data layer; dynamic features connect to PostgreSQL when configured.

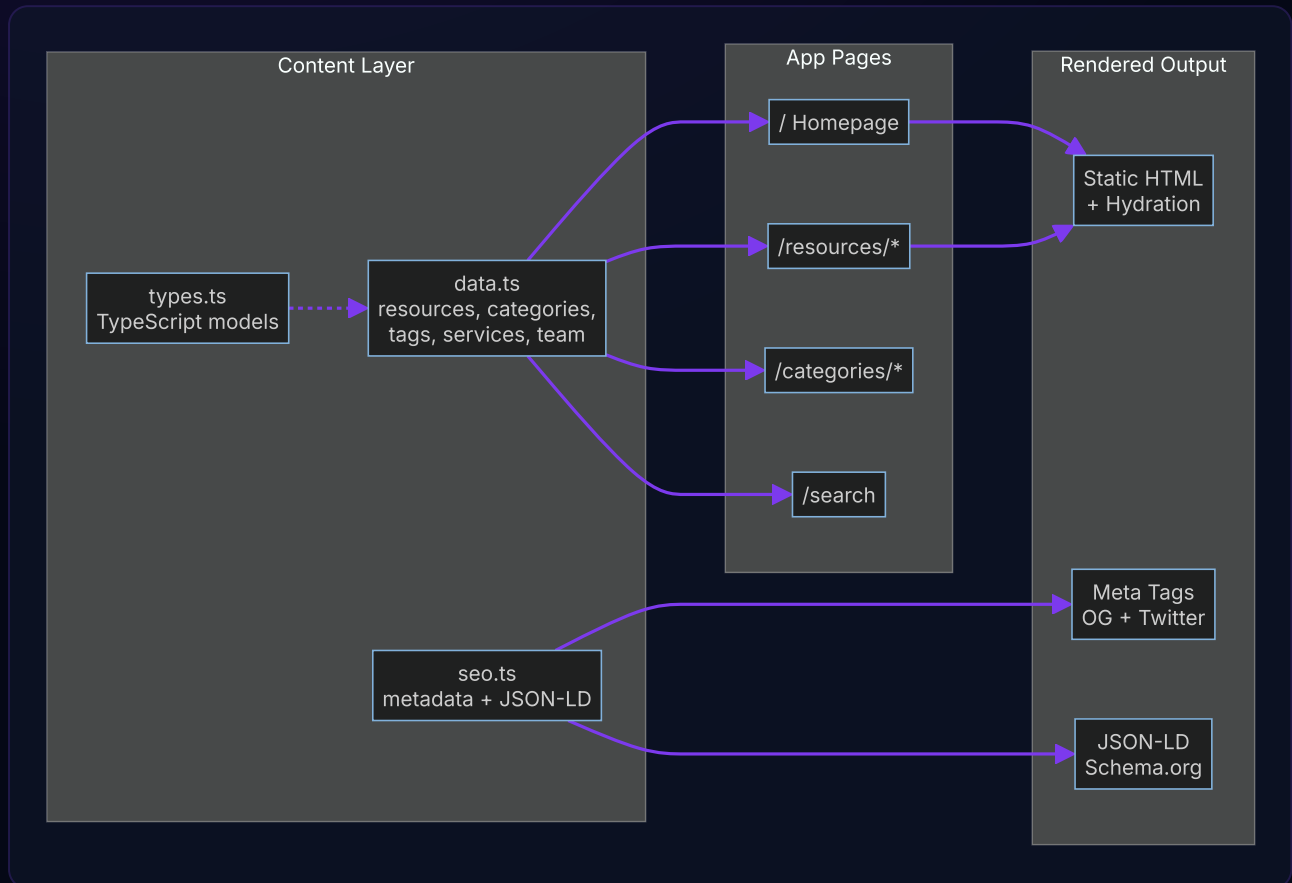


Request Lifecycle

1. User navigates to a route — Next.js resolves App Router page or layout
2. Server Components fetch data from `src/lib/data.ts` (or Prisma when DB connected)
3. Static pages pre-render at build time via `generateStaticParams`
4. Client components hydrate for interactive features (search filters, forms, AI chat)
5. Admin routes pass through middleware — unauthenticated users redirect to login

3. System Architecture (continued)

Data flows from the content layer through page components to the rendered HTML. The AI chat path is fully self-contained and requires no external API keys.



Rendering Strategy

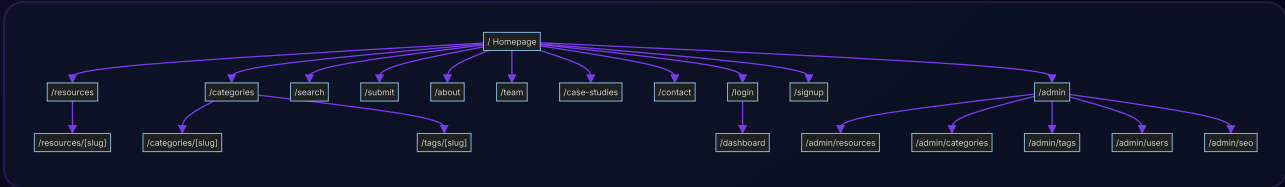
Route Pattern	Strategy	Data Source
<code>/</code> , <code>/about</code> , <code>/contact</code>	Static	<code>data.ts</code>
<code>/resources/[slug]</code>	SSG (<code>generateStaticParams</code>)	<code>data.ts</code>
<code>/categories/[slug]</code>	SSG	<code>data.ts</code>
<code>/search</code>	Client-side filter	<code>data.ts</code> (bundled)
<code>/admin/*</code>	SSR + middleware	<code>data.ts</code> / Prisma
<code>/dashboard</code>	SSR	Session + Prisma

Middleware Scope

Middleware runs only on `/admin/:path*` routes. It checks the session role and redirects non-admin users to the login page with a callback URL. All public routes are unaffected.

4. Site Map

The application exposes public marketing pages, resource directory routes, user account pages, and a protected admin area.



Static Routes (sitemap.xml)

[/](#)[/resources](#)[/categories](#)[/search](#)[/submit](#)[/about](#)[/team](#)[/case-studies](#)[/contact](#)

Dynamic Routes

- [/resources/\[slug\]](#) — Individual resource or service landing page
- [/categories/\[slug\]](#) — Category-filtered resource listing
- [/tags/\[slug\]](#) — Tag-filtered resource listing

Sitemap generation is handled by `src/app/sitemap.ts` and includes all static routes plus every resource, category, and tag URL with appropriate priority and change frequency.

5. Service Pages

Agency services are modeled as resources with rich landing page content. When a resource slug matches a `servicePages` entry in `data.ts`, the detail route renders `ServiceLandingPage` instead of the standard resource layout.

Slug	Service	Route
<code>technical-seo-audit</code>	Technical SEO Audit	<code>/resources/technical-seo-audit</code>
<code>seo-content-writing</code>	SEO Content Writing	<code>/resources/seo-content-writing</code>
<code>da-dr-boost</code>	DA/DR Boost Services	<code>/resources/da-dr-boost</code>
<code>web-design-development</code>	Web Design & Development	<code>/resources/web-design-development</code>
<code>local-seo</code>	Local SEO Services	<code>/resources/local-seo</code>
<code>editorial-guest-posting</code>	Editorial Guest Posting	<code>/resources/editorial-guest-posting</code>
<code>editorial-link-building</code>	Editorial Link Building	<code>/resources/editorial-link-building</code>
<code>white-label-link-building</code>	White Label Link Building	<code>/resources/white-label-link-building</code>

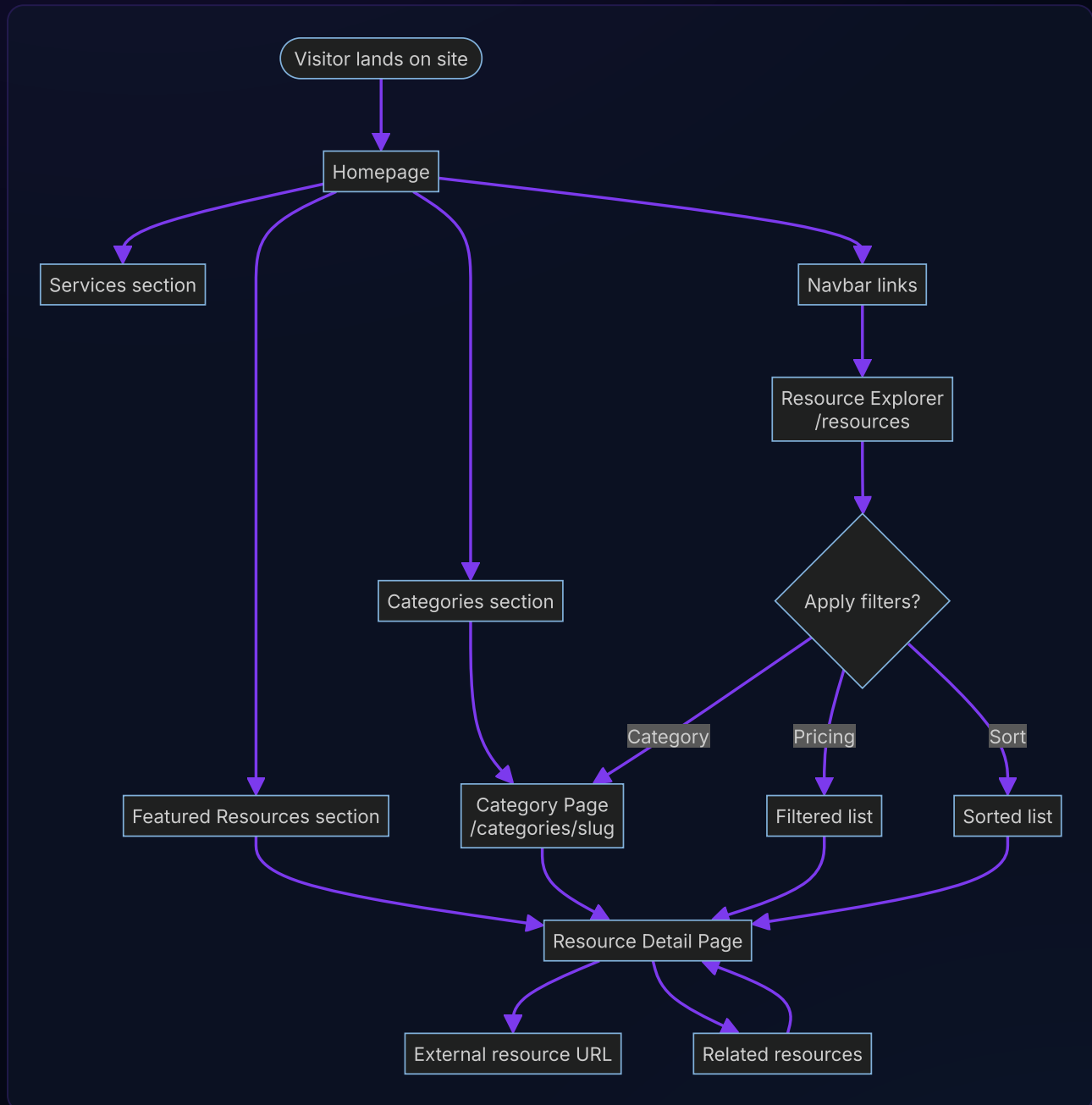
Service Landing Page Sections

Hero Eyebrow, title, subtitle, bullet points, and primary CTA.	Stats Bar Key performance metrics with delta indicators.
Packages Tiered pricing cards with feature lists.	Why Choose Us Value proposition with supporting points.
Process Step-by-step delivery workflow.	Deliverables Concrete outputs clients receive.
FAQ Service-specific questions and answers.	Case Studies Related success stories filtered by category.

Each service page injects `Service` JSON-LD schema via `serviceJsonLd()` for enhanced search visibility.

6. User Flows — Browse & Discover

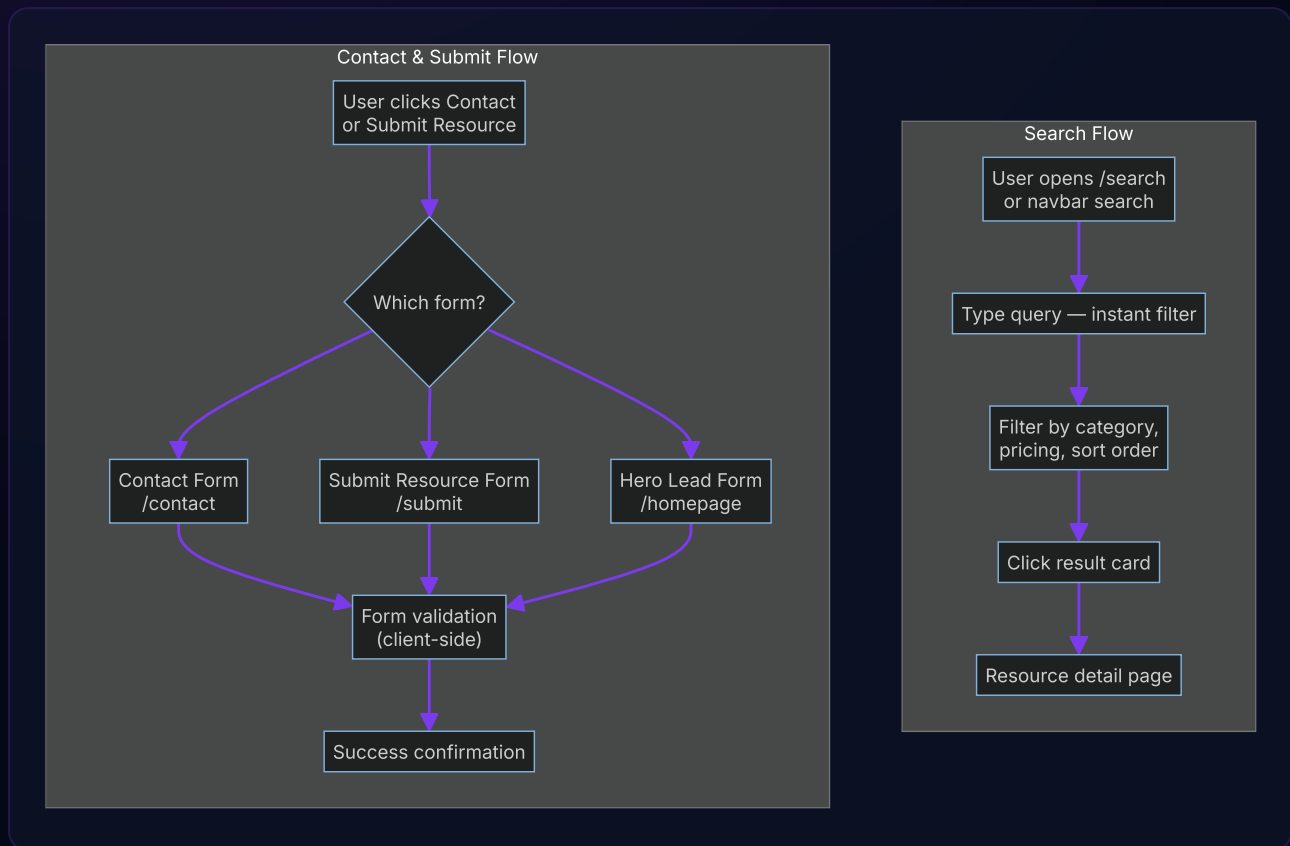
Visitors explore the resource directory through multiple entry points: homepage sections, category pages, tag pages, and the resource explorer.



Homepage Sections

- **Hero** — Animated typing, lead capture form, particle background
- **Featured Resources** — Curated picks linking to detail pages
- **Categories** — Six category cards (Link Building, SEO Tools, Content, Web Dev, Design, AI)
- **Services** — Agency service cards with links to service landings
- **Case Studies, Reviews, FAQ, Pricing, CTA** — Full agency content preserved

6. User Flows — Search & Contact



Search Features

- Real-time text search across title, tagline, description, category, and tags
- Category dropdown filter and pricing tier filter (Free / Freemium / Paid)
- Sort options: relevance, rating, views, newest
- Implemented in `ResourceExplorer` client component

Contact & Lead Capture

- **Contact page** — Full contact form with name, email, message
- **Submit page** — Resource submission form (title, URL, category, description)
- **Hero lead form** — Goal selector on homepage for consultation requests
- **Consultation banner** — Persistent CTA linking to contact

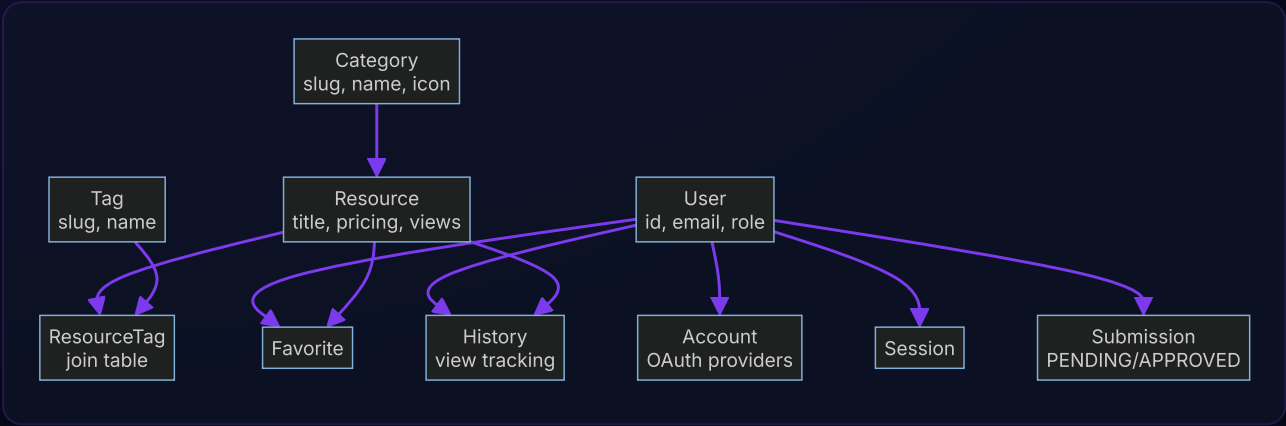
7. Data Layer

All site content lives in `src/lib/data.ts` — a single source of truth preserving the original linksresource.com content. TypeScript types in `src/lib/types.ts` enforce structure.



8. Prisma Models

The database schema supports user accounts, resource management, favorites, view history, and resource submissions. Defined in `prisma/schema.prisma`.



Enums

Enum	Values	Usage
<code>Role</code>	USER, ADMIN	User access level
<code>Pricing</code>	FREE, FREEMIUM, PAID	Resource pricing tier
<code>SubmissionStatus</code>	PENDING, APPROVED, REJECTED	Submission workflow

8. Prisma Models (continued)

Model Reference

Model	Key Fields	Relations
User	email, passwordHash, role	accounts, sessions, favorites, history, submissions
Account	provider, providerAccountId	user (OAuth)
Session	sessionToken, expires	user
Category	slug, name, description, icon, gradient	resources[]
Tag	slug, name	resources (M2M via ResourceTag)
Resource	slug, title, pricing, featured, rating, views	category, tags, favorites, history
ResourceTag	resourceId, tagId	Composite PK join table
Favorite	userId, resourceId	Unique per user+resource
History	userId, resourceId, viewedAt	View tracking index on userId
Submission	title, url, status	Optional user relation

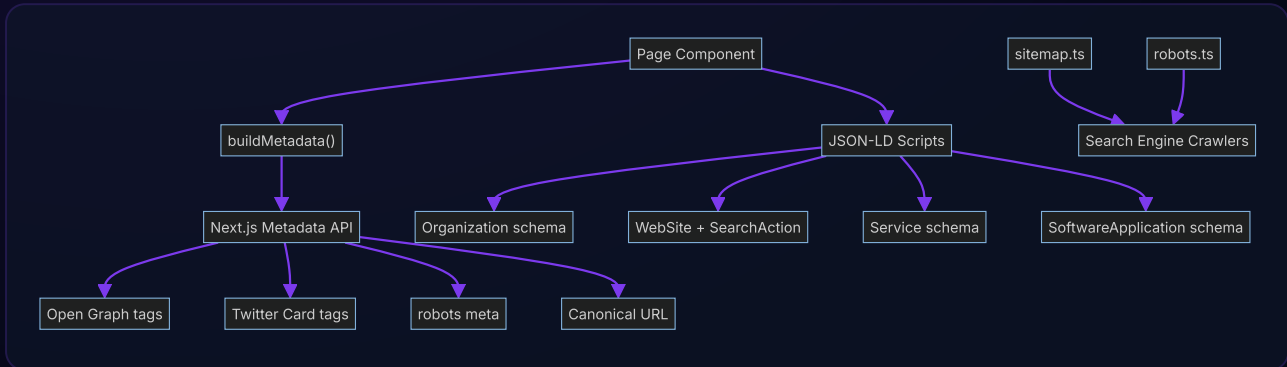
Database Setup Commands

```
npm run db:up      # Start Postgres (Docker, port 5433)
npm run db:setup   # Docker + push schema + seed
npm run db:push    # Push schema to database
npm run db:seed    # Seed admin user + content
npm run db:studio  # Open Prisma Studio GUI
```

Connection string: `postgresql://postgres:postgres@localhost:5433/links_resource`

9. SEO

SEO is handled centrally through `src/lib/seo.ts` with per-page metadata, structured data, and crawl configuration.

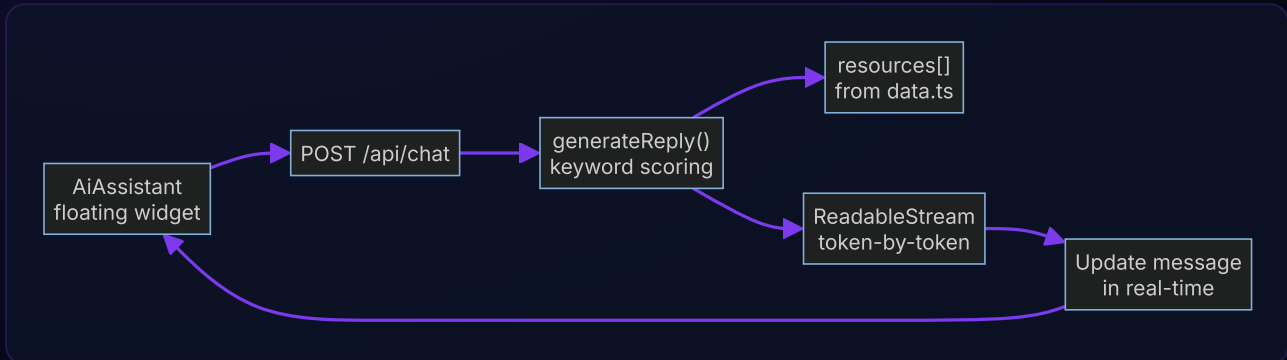


SEO Features

- **Dynamic metadata** — Title, description, keywords per page via `buildMetadata()`
- **Open Graph + Twitter** — Large image cards using `/og.png` (1200×630)
- **JSON-LD** — Organization, WebSite (with SearchAction), Service, SoftwareApplication schemas
- **Sitemap** — Auto-generated at `/sitemap.xml` with priorities and change frequencies
- **Robots** — Allow indexing on public pages; `noindex` on admin via `buildAdminMetadata()`
- **Google verification** — HTML tag via `GOOGLE_SITE_VERIFICATION` env var
- **Dynamic icons** — `icon.tsx` and `apple-icon.tsx` generate favicons at build time

10. AI Chat Assistant

A floating AI assistant widget provides resource recommendations via a streaming chat interface. It works out of the box with zero API keys using a built-in recommendation engine.



Recommendation Engine

1. Extract meaningful keywords from user message (stop words removed)
2. Score each resource by title (+3), category (+2), tags (+2), description (+1) matches
3. Return top 3 matches sorted by score, then views
4. Special handling for category queries and fallback to featured resources

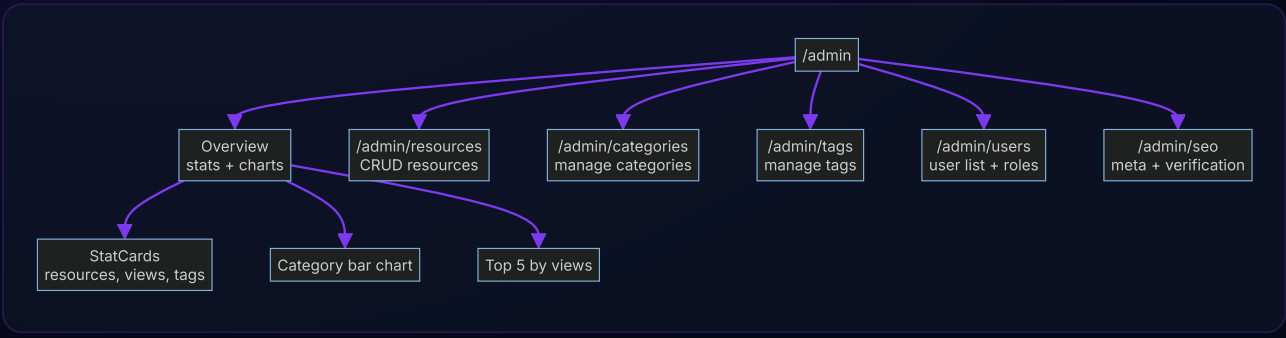
Client Component (`AiAssistant`)

- Persistent floating button with sparkle icon — opens chat panel
- Pre-filled suggestion chips: "Best link building tools", "Recommend SEO resources", etc.
- Streaming response rendered token-by-token via `ReadableStream` reader
- Framer Motion animations for panel open/close and message appearance

To swap in a real LLM, set `OPENAI_API_KEY` and replace `generateReply()` with a streamed provider call — the response shape and streaming interface remain identical.

11. Admin Dashboard

The admin area at `/admin` provides analytics and content management. Access is restricted to users with the ADMIN role via middleware.



Admin Pages

Route	Purpose
<code>/admin</code>	Dashboard overview — resource count, category breakdown, top viewed
<code>/admin/resources</code>	Resource listing and management
<code>/admin/categories</code>	Category management
<code>/admin/tags</code>	Tag management
<code>/admin/users</code>	User accounts and role assignment
<code>/admin/seo</code>	SEO settings and Google Search Console verification

All admin pages use `buildAdminMetadata()` to set `noindex` and prevent search engine indexing.

12. API Routes

Server-side API endpoints handle chat streaming, user registration, and session management.

Route	Method	Runtime	Description
<code>/api/chat</code>	POST	nodejs	Streaming AI chat — accepts <code>{ messages[] }</code> , returns text/plain stream
<code>/api/register</code>	POST	nodejs	User registration — validates input, hashes password, creates User record
<code>/api/auth/[...nextauth]</code>	GET/POST	edge	NextAuth v5 handlers — sign in, sign out, session, CSRF

Server Actions

Action	File	Description
<code>submitResource</code>	<code>app/actions/submit-resource.ts</code>	Creates a Submission record with PENDING status

`/api/chat` Request Flow

1. Client sends POST with conversation history as JSON
2. Server extracts last user message
3. `generateReply()` scores resources and builds response text
4. Response tokens streamed with 18ms delay for typing effect
5. Content-Type: `text/plain; charset=utf-8`

`/api/register`

- Validates name, email, password with Zod schema
- Checks for duplicate email before creation
- Hashes password with bcryptjs
- Creates User with default USER role

13. Components

Components are organized by domain under `src/components/`. UI primitives follow shadcn-style patterns with Tailwind variants.

Layout

- `navbar.tsx` — Main navigation + search link
- `footer.tsx` — Site links, socials, legal
- `scroll-progress.tsx` — Reading progress bar
- `theme-toggle.tsx` — Dark/light mode switch
- `hash-scroll.tsx` — Anchor scroll handler

Home

- `hero.tsx` — Full-screen animated hero
- `typing-text.tsx` — Rotating headline effect
- `featured-resources.tsx` — Resource card grid
- `categories-section.tsx` — Category cards
- `services-section.tsx` — Agency services
- `how-it-works.tsx` — Process steps
- `case-studies.tsx` — Success stories
- `reviews.tsx` — Client testimonials
- `faq.tsx` — Accordion FAQ
- `pricing.tsx` — Package tiers
- `cta.tsx` — Call to action banner
- `marquee.tsx` — Logo/partner scroll

Resources

- `resource-explorer.tsx` — Search + filter UI
- `resource-card.tsx` — Directory entry card

Services

- `service-landing-page.tsx` — Full agency service page

AI

- `ai-assistant.tsx` — Floating chat widget

Forms

- `contact-form.tsx` — Contact page form
- `submit-form.tsx` — Resource submission
- `hero-lead-form.tsx` — Homepage lead capture
- `auth-form.tsx` — Login/signup form

Backgrounds

- `aurora-background.tsx` — Gradient aurora effect
- `particles.tsx` — Interactive particle field

UI Primitives

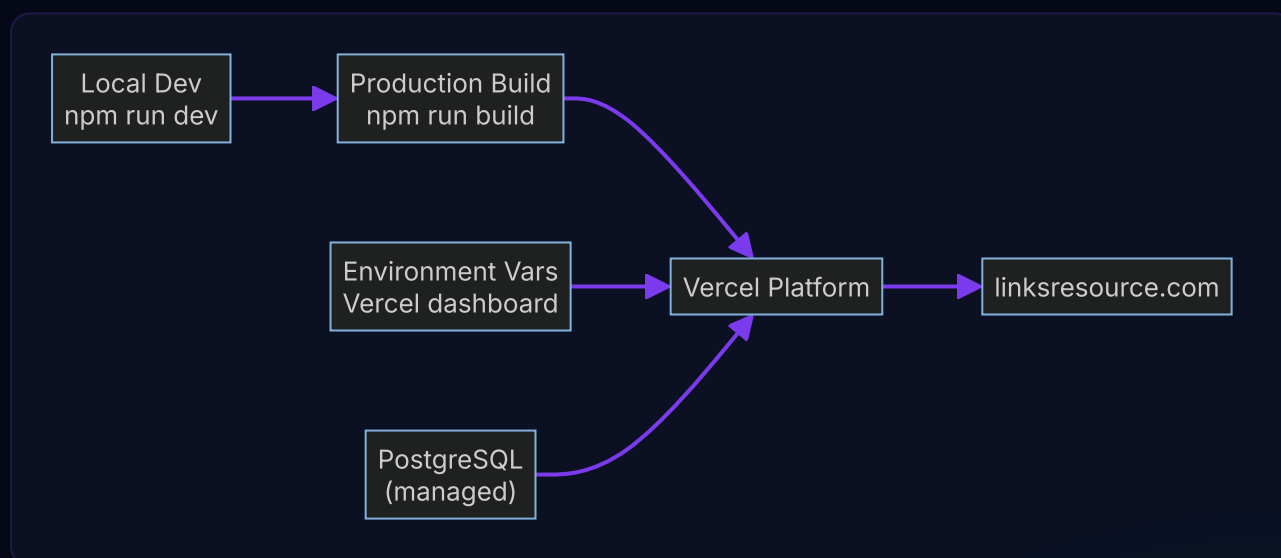
- `button.tsx`, `card.tsx`, `badge.tsx`, `input.tsx`

14. Environment Variables

Variable	Required	Purpose
<code>DATABASE_URL</code>	For auth/DB	PostgreSQL connection string
<code>AUTH_SECRET</code>	For auth	NextAuth session encryption (<code>npx auth secret</code>)
<code>NEXT_PUBLIC_SITE_URL</code>	Recommended	Canonical URL for SEO metadata
<code>SEED_ADMIN_PASSWORD</code>	For seeding	Admin password for <code>npm run db:seed</code>
<code>GOOGLE_SITE_VERIFICATION</code>	Optional	Google Search Console HTML tag value
<code>AUTH_GOOGLE_ID/SECRET</code>	Optional	Google OAuth provider credentials
<code>AUTH_GITHUB_ID/SECRET</code>	Optional	GitHub OAuth provider credentials

15. Deployment

Links Resource is designed for deployment on Vercel with optional PostgreSQL (Vercel Postgres, Neon, or Supabase).



Production Checklist

1. Set `NEXT_PUBLIC_SITE_URL=https://linksresource.com` in Vercel env
2. Generate and set `AUTH_SECRET`
3. Configure `DATABASE_URL` pointing to production PostgreSQL
4. Run `npm run db:push && npm run db:seed` against production DB
5. Add custom domain in Vercel → Project Settings → Domains
6. Set `GOOGLE_SITE_VERIFICATION` and submit sitemap to Search Console
7. Deploy: `npm run build && npm run start` (or push to Git for Vercel auto-deploy)

16. Roadmap

Phase	Feature	Status
1	Core site rebuild with data layer	Complete
1	Resource directory + search + filters	Complete
1	Service landing pages (7 services)	Complete
1	AI chat assistant (recommendation engine)	Complete
1	SEO: metadata, JSON-LD, sitemap, OG	Complete
1	Admin dashboard + middleware guard	Complete
1	Team page + case studies	Complete
2	Full Prisma-backed resource CRUD in admin	Planned
2	User favorites + view history (dashboard)	Planned
2	Real LLM integration for AI assistant	Planned
2	Submission approval workflow in admin	Planned
3	Analytics integration (Plausible / GA4)	Planned
3	Email notifications for submissions	Planned
3	Multi-language support (i18n)	Future

17. File Structure

```
links-resource/
├─ prisma/
│  ├─ schema.prisma      # Database models
│  └─ seed.ts            # Seed script
├─ public/
│  ├─ og.png              # Open Graph image
│  └─ team/               # Team member photos
├─ scripts/
│  ├─ db-setup.mjs        # One-command DB setup
│  ├─ load-env.mjs        # Env loader utility
│  └─ run-with-env.mjs    # Run commands with .env
├─ docs/
│  ├─ project-documentation.html
│  └─ generate-pdf.mjs    # Puppeteer PDF generator
├─ src/
│  ├─ app/                # App Router pages & API
│  │  ├─ admin/           # Admin dashboard routes
│  │  ├─ api/             # chat, auth, register
│  │  ├─ actions/         # Server actions
│  │  ├─ resources/       # Directory + detail pages
│  │  ├─ categories/      # Category pages
│  │  ├─ tags/            # Tag pages
│  │  ├─ search/          # Search page
│  │  ├─ team/            # Team page
│  │  └─ sitemap.ts        # Dynamic sitemap
│  └─ lib/                # Common utilities
```

